

FUNDAMENTALS OF PROGRAMMING

MOCK FINAL EXAM

Faculty of Information Technology
University of Science, VNU-HCM

Time Allotted: 90 Minutes

Instructions

- **Language:** C++
 - **Allowed Libraries:** `<iostream>`, `<iomanip>`, `<fstream>`, `<cstring>`, `<cctype>`, `<cmath>`.
 - **Forbidden Libraries:** `<string>`, `<vector>`, `<algorithm>`.
 - **Memory Management:** You must manage memory manually. All new allocations must have a corresponding delete.
 - **Strings:** All strings must be handled as C-strings (`char*` or `char[]`).
-

Part I: Flow of Control & Functions (3 Questions)

Q1. The Perfect Square Check

Write a function `bool isPerfectSquare(int n)` that returns `true` if n is a perfect square (e.g., 4, 9, 16), and `false` otherwise.

- *Constraint:* Do not use `sqrt` from `<cmath>`. Use a loop structure.
- **Input:** An integer n ($0 \leq n \leq 10^9$).
- **Output:** `true` or `false`.

Q2. Symmetrical Number

Write a function to check if a positive integer n is a symmetrical number (palindrome).

- **Input:** Integer n ($n > 0$).
- **Examples:** $121 \rightarrow \text{true}$; $123 \rightarrow \text{false}$.

Q3. Recursive Sum of Factorials

Write a **recursive** function to calculate the sum S :

$$S = 1! + 2! + \dots + n!$$

- **Input:** Integer n ($1 \leq n \leq 10$).
- **Output:** The calculated integer sum.

Part II: Arrays & C-Strings (3 Questions)

Q4. Diagonal Sums

Given a square matrix A of size $N \times N$. Write a function to calculate the sum of the main diagonal and the anti-diagonal.

- **Input:** Integer N , followed by the matrix elements.
- **Output:** Two integers: (Sum Main, Sum Anti).

Q5. String Normalization

Write a function `void normalize(char* s)` that transforms a sentence such that the first letter of each word is capitalized, and all other letters are lowercase. Words are separated by a single space.

- **Example:** "hEllo wORLD" $\rightarrow$ "Hello World"

Q6. Find Element in 2D Array

Write a function that takes a 2D array (represented as `int a[][100]`) with m rows and n columns, and an integer k . Return the pair of indices (r, c) of the first occurrence of k . If k is not found, return $(-1, -1)$.

Part III: Pointers & Memory Management (3 Questions)

Q7. Dynamic Array Resizing

Write a function `int* resizeArray(int* arr, int currentSize, int newSize)` that:

1. Allocates a new dynamic array of size `newSize`.
2. Copies data from the old array to the new one (up to the minimum of the two sizes).
3. Fills any excess cells in the new array with 0.
4. Deletes the old array to prevent memory leaks.
5. Returns the pointer to the new array.

Q8. Pointer Swap

Implement `void swap(int* a, int* b)` to swap the values of two integers using pointers. Write a main function to demonstrate swapping two variables $x = 5, y = 10$.

Q9. Dynamic 2D Array Allocation

Write a function `int** createMatrix(int rows, int cols)` that allocates a 2D array using pointers to pointers. You must allocate the array of row pointers first, then allocate columns for each row.

Part IV: Structures (3 Questions)

Q10. The Point Structure

Define a struct `Point` with fields x, y (float). Write a function:

`float distance(Point p1, Point p2)`

that returns the Euclidean distance between them: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Q11. Student Management

Define a struct `Student` containing:

```
1 struct Student {
2     char id[10];
3     char name[50];
4     float score;
5 };
```

Write a function `Student findBestStudent(Student list[], int n)` that returns the student struct with the highest score. **Q12. Fraction Reduction**

Define a struct `Fraction` with integers `numerator` and `denominator`. Write a function `void reduce(Fraction &f)` that simplifies the fraction to its lowest terms. (e.g., $2/4 \rightarrow 1/2$).

Part V: Files (2 Questions)

Q13. Text File Integer Processing

Write a complete program that:

1. Reads `input.txt`. The first line is an integer N . The following lines contain N integers.
2. Computes the sum of all **even** numbers.
3. Writes the result to `output.txt`.

```
1 Input (input.txt):  
2 3  
3 1 2 4  
4  
5 Output (output.txt):  
6 6
```

Listing 1: Example I/O

Q14. Text File Word Count

Write a program that:

1. Opens a text file named `essay.txt`.
2. Counts the total number of words in the file. A "word" is defined as any sequence of characters separated by whitespace (spaces, tabs, or newlines).
3. Writes the final count to `summary.txt`.

```
1 Input (essay.txt):  
2 Hello world  
3 Programming is fun  
4  
5 Output (summary.txt):  
6 5
```

Listing 2: Example I/O

Part VI: Linked Lists (1 Question)

Q15. Linked List Operations

Given the node structure:

```
1 struct Node {  
2     int data;  
3     Node* next;  
4 };
```

Write a function `void removeValues(Node* &head, int val)` that removes **all** nodes in the list that have `data == val`.

- Ensure you handle the case where the **head** node itself must be removed.
- Ensure memory is correctly freed using `delete`.